

Korg.jl Feature Exploration Modules

A Reference Guide

20 self-contained Julia example scripts covering Korg.jl's spectral synthesis features, written while learning the package hands-on -- plus a consolidated list of API details that differed from what the scripts initially assumed, in case it's useful for future tutorials, examples, or documentation.

Compiled by Joseph Karpinski, with debugging assistance from Claude (Anthropic) • September 2026 • Korg.jl package hash rR21M, Julia 1.12.7, macOS

Why this document exists

Korg.jl is a capable 1D LTE stellar spectral synthesis package. To learn what it offers hands-on, I wrote 20 small, self-contained "feature exploration" scripts, each demonstrating one Korg capability (basic synthesis, custom abundances, equivalent-width fitting, MARCS atmosphere handling, NLTE-style corrections, APOGEE-resolution synthesis, contribution functions, and more).

Along the way, most of the scripts needed adjustments to match the installed package's current API -- function names, keyword arguments, constructor signatures, and return shapes that have evolved since some of the examples I started from were written. A few of these are easy to miss because they don't raise an error at all; they just quietly produce a plausible-looking but physically wrong result. That pattern is collected in the "Known API Gotchas" section below, offered in case it's a useful reference for the Korg.jl maintainers and community -- as a starting point for expanded tutorials and examples, or simply as a worklist of places where usage has drifted from what's shown in older material.

Each module referenced below is included as a separate, runnable `.jl` file alongside this PDF.

The 20 Feature Modules

1. Basic LTE synthesis

`korg_basic_lte_synthesis.jl`

The starting point: `Korg.synth()` over a wavelength range with a VALD solar linelist. Establishes the recurring fixes needed almost everywhere else: `get_linelist(:vald)` doesn't exist (use `get_VALD_solar_linelist()`), and `synth` takes `wavelengths=(λmin,λmax)` rather than `λmin=/λmax=` keywords.

2. Interpolated MARCS atmosphere synthesis

`korg_marcs_atmosphere_synthesis.jl`

Builds an atmosphere via `Korg.interpolate_marcs(Teff, logg, A_X)` rather than letting `synth()` build one implicitly, then synthesizes from it directly with the low-level `Korg.synthesize()`. Establishes that atmosphere objects expose no flat `.depth/.T` arrays -- data lives in `.layers`, with `Korg.get_temps(atm)` as the sanctioned temperature accessor.

3. Custom abundance-pattern synthesis

`korg_custom_abundance_synthesis.jl`

Demonstrates per-element abundance overrides on `synth` (e.g. a carbon-enhanced, metal-poor pattern). Key trap: individual element keywords must be splatted from a Dict with a LEADING SEMICOLON (`synth(; Teff=..., abund...)`) or Julia parses the splat as positional Pair objects and throws a `MethodError`.

4. Abundance from a measured equivalent width

`korg_ew_abundance_fitting.jl`

Fits a single-line abundance from an EW measurement. `Korg.fit_abundance_from_EW` does not exist; the real function is `Korg.Fit.ews_to_abundances(atm, linelist, A_X, EWs)`, which returns a plain `Vector{Float64}`, not the documented (abundances, slopes) tuple.

5. EW fitting round-trip validation

`korg_ew_roundtrip_validation.jl`

A methodology module: generates self-consistent synthetic EWs from a known atmosphere via `Korg.Fit.calculate_EWs`, then confirms `Korg.Fit.ews_to_stellar_parameters` recovers the known truth from an offset initial guess. Useful as a template for telling "code bug" apart from "physically inconsistent input data" in any future EW-fitting work.

6. Excitation/ionization balance

`korg_excitation_ionization_balance.jl`

Solves for stellar parameters from a small Fe I/II line list via `Korg.Fit.ews_to_stellar_parameters` (`Korg.solve_excitation_ionization_balance` does not exist). Requires the linelist sorted by wavelength ahead of the call.

7. Line-by-line Fe abundance scatter

`korg_line_by_line_fe_abundances.jl`

Computes per-line Fe abundances from several EWs in one `Korg.Fit.ews_to_abundances` call (there is no per-line convenience wrapper), and reports the scatter across lines as a diagnostic of input-data self-consistency.

8. Automatic differentiation through synthesis

`korg_ad_spectrum_derivative.jl`

Confirms ForwardDiff.jl differentiates cleanly through Korg's `synth()` and its internal atmosphere interpolation ($d(\text{flux})/d(\text{Teff})$). Also fixes a performance trap: the linelist must be loaded ONCE outside the function being differentiated, not reloaded on every ForwardDiff evaluation.

9. Full-spectrum χ^2 parameter fitting

`korg_full_spectrum_chi2_fit.jl`

Fits $\text{Teff}/\log g/[M/H]/v_{\text{mic}}/v_{\text{sini}}$ directly from a synthetic noisy spectrum via `Korg.Fit.fit_spectrum`. Several real traps: `obs_err` and `fixed_params` are required positional arguments with no defaults; the resolution keyword `R` has NO default and must be a genuinely finite number ($R=\text{Inf}$ propagates NaNs into the Levenberg-Marquardt solver's linear algebra). Also demonstrates extracting parameter uncertainties from the returned covariance matrix.

10. Custom (non-VALD) linelist loading

`korg_custom_linelist_loading.jl`

Loads a hand-supplied MOOG-format linelist. Uncovered a likely genuine bug in Korg's own built-in MOOG parser (`parse_moog_linelist` throws `MethodError: no method matching +{::Nothing, ::Int64}` on a valid whitespace-separated MOOG file -- reported upstream). Worked around by bypassing Korg's format parser entirely and constructing `Korg.Line/Korg.Species` objects directly from a hand-rolled parser.

11. Reading a raw MARCS .mod file from disk

`korg_marcs_file_parsing.jl`

Reads a real downloaded MARCS atmosphere file (`Korg.read_marcs` does not exist; use `Korg.read_model_atmosphere`) and synthesizes from it. Also catches a subtle silent-correctness bug: MARCS 'standard' grid files below solar metallicity bake in alpha-element enhancement (encoded in the filename, e.g. `..._z-1.00..._a+0.40...`), so the abundance vector fed to synthesis must include that alpha enhancement too, or line strengths for O/Mg/Si/Ca/Ti silently desync from the atmosphere that actually produced them.

12. Custom solar abundance scale

`korg_custom_solar_scale.jl`

Two-stage fix. First attempt used `synth`'s per-element keywords as absolute $A(X)$ values -- but those keywords are OFFSETS added on top of the metallicity-scaled default, not substitutions, and this blew Korg's internal $[M/H]$ inference far outside the MARCS grid range. Fixed by building the full 92-element $A(X)$ vector by hand and calling the low-level `Korg.interpolate_marcs` with an explicit `solar_abundances=` reference vector.

13. Microturbulence + macroturbulence broadening

`korg_vmic_vmac_broadening.jl`

`vmic` and `vsini` are real `synth()` keywords; `vmac` is not -- Korg has no built-in macroturbulence mechanism, since it's conventionally just a post-hoc Gaussian smoothing of the emergent spectrum. Implements that convolution by hand.

14. Applying a user-supplied NLTE correction

`korg_nlte_correction.jl`

Fits an LTE abundance from an EW, applies a user NLTE correction function, then re-synthesizes with the corrected abundance. Combines several of the fixes above: the real `Fit.ews_to_abundances` signature, and the absolute-vs-offset abundance trap from the custom-solar-scale module.

15. High-resolution APOGEE (R~22,500) synthesis

`korg_apogee_synthesis.jl`

Builds the APOGEE log- λ grid and LSF matrix, then synthesizes and resamples a giant-star spectrum across the full H band. Fix: `compute_LSF_matrix` needs the synthesis's own native wavelength array as its source grid, so it must be built AFTER `synth()` runs, not before, and not from a bare $(\lambda_{\min}, \lambda_{\max})$ bounds tuple.

16. Synthetic photometry / AB magnitude

`korg_synthetic_photometry.jl`

Integrates a synthesized spectrum against a filter transmission curve to compute a synthetic magnitude. Catches a bug that would NOT have raised an error: `synth()`'s flux is continuum-rectified (dimensionless, ~1 outside lines) by default, not a physical flux density -- feeding that directly into an AB-magnitude zeropoint calculation silently produces a meaningless number. Fixed by reconstructing physical flux as `flux .* continuum` before the photometric integral.

17. Finite-difference abundance response function

`korg_abundance_response.jl`

Computes $\partial \text{Flux} / \partial A(\text{Fe})$ by synthesizing at two nearby Fe abundances against ONE shared atmosphere. Reuses the absolute-A(X) construction from the custom-solar-scale fix, and holds the atmosphere fixed across both syntheses so the derivative reflects only the abundance's effect on opacity.

18. Small synthetic spectral grid

`korg_spectral_grid_generation.jl`

Loops `synth()` over a $T_{\text{eff}} \times \log g \times [\text{M}/\text{H}] \times v_{\text{mic}}$ grid (24 combinations), loading the linelist once outside the loop for performance.

19. Depth-dependent contribution function

`korg_contribution_function.jl`

Korg has no built-in `contribution_function` call -- this was built from first principles (LTE Planck source function + the per-layer opacity exposed by `synthesize()`'s `result.alpha`). Two real bugs surfaced and were fixed along the way: getting the row/column axis of `result.alpha` backwards, and a ~14-decade optical-depth range from a hand-integrated depth scale with a unit error -- resolved by using the MARCS model's own pre-calibrated `tau_ref` instead of manually integrating opacity over height.

20. Off-grid atmosphere without a custom file grid

`korg_custom_atmosphere_grid.jl`

The original version tried to glob a directory of MARCS files into a Dict and call a nonexistent `Korg.interpolate_atmosphere(grid, ...)` over it -- Korg has no public API for interpolating over a user-assembled atmosphere collection. Rewritten to use Korg's actual interpolation function, `Korg.interpolate_marcs`, which interpolates against Korg's own bundled MARCS grid directly, with no file I/O needed at all.

Known Korg API Gotchas

Every one of these was hit while working through the modules above, against Korg.jl package hash rR21M on Julia 1.12.7. Several are simple naming drift from older tutorials; a few are silent-correctness issues that produce no error at all, which seem worth flagging separately since they're the easiest to miss.

get_linelist(:vald) does not exist

The function referenced in tutorials for loading a standard linelist isn't part of the installed API. Use `Korg.get_VALD_solar_linelist()`, or the survey-specific getters (`get_APOGEE_DR17_linelist`, `get_GALAH_DR3_linelist`, `get_GES_linelist`), or `read_linelist(path; format=...)` for a custom file.

synth() has no $\lambda_{\min}$ / $\lambda_{\max}$ keywords

The wavelength range is a single keyword, `wavelengths=( $\lambda_{\min}$ ,  $\lambda_{\max}$ )`.

m_H is not a valid keyword; it's M_H

Confirmed via Korg's own error message: 'm_H is no longer a supported keyword argument... Use M_H instead.'

interpolate_marcs takes positional, not keyword, arguments

`interpolate_marcs(; Teff=, logg=, M_H=)` raises a `MethodError`. The real signature is positional: `interpolate_marcs(Teff, logg, A_X_or_M_H=0, alpha_m=0, C_m=0; kwargs...)`.

Atmosphere objects expose no .depth / .T / .Pgas

Per-layer data lives in `atm.layers` (a vector of layer structs with `.tau_ref`, `.z`, `.temp`, `.electron_number_density`, `.number_density`). `Korg.get_temps(atm)` is the sanctioned temperature accessor; there is no built-in gas-pressure accessor at all (derive it via the ideal gas law if needed).

synth() has no atmosphere= keyword

`synth()` always builds its own atmosphere internally from `Teff/logg/M_H`. To synthesize from an atmosphere you built or read yourself, drop to the low-level `Korg.synthesize(atm, linelist, A_X, wavelengths)`, which returns UN-rectified physical flux (rectify manually via `flux ./ cntm` if you want a normalized spectrum).

synth() has no abundances=Dict(...) keyword

Individual elements are separate keywords (e.g. `C=1.2`). Splatting a `Dict` into them REQUIRES a leading semicolon -- `synth(; Teff=..., dict=...)` -- or Julia parses the splat as positional `Pair` arguments and raises a `MethodError`.

Per-element synth() keywords are OFFSETS, not absolute A(X) values

This is the trap most likely to silently corrupt a result rather than error: passing `Fe=7.50` does not set `A(Fe)` to 7.50, it adds +7.50 dex on top of the existing (metallicity-scaled) abundance. This blows Korg's internal `[M/H]` inference (used to pick a MARCS grid point) far out of range if you pass anything that looks like an absolute abundance. To set true absolute `A(X)` values, build the full 92-element `A(X)` vector by hand (starting from `Korg.format_A_X()` as the default reference) and use the low-level `interpolate_marcs/synthesize` path, passing your reference vector as `solar_abundances=` if calling `interpolate_marcs` directly.

Korg.Line() has no keyword constructor

`Line( $\lambda$ =, species=, loggf=,  $\chi$ =)` raises a `MethodError`. The constructor is strictly positional: `Line(wl_angstrom, log_gf, species, E_lower)`, and species must be wrapped as `Korg.Species("Fe I")` -- a bare string is rejected.

Korg.fit_abundance_from_EW and Korg.solve_excitation_ionization_balance do not exist

The real functions live in the `Korg.Fit` submodule: `Korg.Fit.ews_to_abundances(atm, linelist, A_X, EWs)` and `Korg.Fit.ews_to_stellar_parameters(linelist, EWs; Teff0=, logg0=, vmic0=, M_H0=, ...)`. The linelist must be pre-sorted by wavelength for the latter.

Fit.ews_to_abundances returns a plain Vector, not a tuple

Documentation for some versions describes a (abundances, slopes) return; the installed version returns a single Vector{Float64}. Deconstructing as abundances, slopes = ... raises a BoundsError on the 1-element result.

Fit.fit_spectrum requires obs_err and fixed_params, and R has no default

Both are positional arguments with no defaults. Separately, the resolution keyword R must be explicitly given and must be a genuinely finite number -- R=Inf propagates NaNs into the LsqFit Levenberg-Marquardt solver's linear algebra (ArgumentError: matrix contains Infs or NaNs). A large finite value (e.g. 300_000.0) works as an effectively-no-broadening stand-in. The returned NamedTuple's covariance field gives parameter uncertainties via sqrt.(diag(covariance_matrix)).

Korg's own built-in MOOG linelist parser has a real bug

read_linelist(path; format="moog") on a valid whitespace-separated MOOG-style file throws MethodError: no method matching +(::Nothing, ::Int64) inside parse_moog_linelist (linelist.jl). Reported upstream; the practical workaround is bypassing Korg's format parser entirely and constructing Korg.Line/Korg.Species objects directly from a hand-rolled line-by-line parser.

Korg.read_marcs does not exist

Reading a MARCS .mod file from disk is Korg.read_model_atmosphere(path).

MARCS 'standard' grid files bake in alpha-enhancement below solar metallicity

The filename convention (e.g. p5000_g+3.0_m0.0_t01_st_z-1.00_a+0.40_..._mod) encodes both [Fe/H] (z) and [alpha/Fe] (a) into the atmosphere's actual physical structure. Building the synthesis abundance vector from [Fe/H] alone silently desyncs O/Mg/Si/Ca/Ti line strengths from the atmosphere that produced them -- no error, just wrong line depths for those elements.

No public Korg API interpolates over a user-supplied atmosphere collection

Korg.interpolate_atmosphere(your_dict, Teff,logg,M_H) does not exist. Korg's real interpolation function, interpolate_marcs, works against its own bundled MARCS grid data, not an arbitrary Dict of atmospheres read from disk.

compute_LSF_matrix needs the synthesis's own native wavelength grid

It must be called AFTER synth()/synthesize() runs, using the real wls array that comes back -- not a ($\lambda_{\min}$, $\lambda_{\max}$) bounds tuple, and not before the synthesis that produces the grid it needs to map from.

vmac is not a real synth() keyword

Korg has no built-in macroturbulence broadening mechanism. vmic and vsini are real (they enter the physical line-formation/rotation calculation); macroturbulence is conventionally applied as a separate post-hoc Gaussian convolution of the emergent spectrum, not something the radiative transfer itself models.

synth()'s flux is continuum-rectified by default -- not a physical flux density

This will not raise an error if misused. flux from synth() is a dimensionless ratio (~1 outside lines), not erg/s/cm²/Å. Anything requiring physical flux (synthetic photometry, absolute magnitudes, flux calibration) needs flux .* cont reconstructed first, or synthesize() called with rectify=false.

All 20 modules referenced above are included as standalone, runnable Julia files alongside this document. Each prints numbered progress messages as it runs and saves its resulting plot as a PNG; several include inline comments explaining the specific fix applied and why, in case that context is useful for anyone extending them further.